Manual de uso — Generador de laboratorios Wi-Fi

Sistema para crear laboratorios de hacking ético Wi-Fi **100 % virtualizados** con `mac80211_hwsim`. A partir de un fichero de configuración, `lab_generator.py` produce un script de despliegue bash autónomo que levanta todos los escenarios (puntos de acceso, clientes y tráfico) aislados en *network namespaces*.

> **Uso exclusivamente educativo y en el aula.** Todas las radios son simuladas: no hay emisión real y las prácticas solo funcionan contra los APs del propio laboratorio. Atacar redes reales sin autorización expresa por escrito es ilegal (en España, arts. 197 bis y 264 del Código Penal).

---

1. Contenido del proyecto

Fichero	Para qué sirve
`lab_generator.py`	Generador. Lee el `.conf` y escribe el script de despliegue.
`lab.conf.example`	Fichero de configuración de ejemplo, listo para copiar y editar.
`despliegue.sh`	Script de despliegue (generado, o el de referencia hecho a mano). Se ejecuta en la VM.
`compila_hostapd_pmkid.sh`	Compila un `hostapd` parcheado necesario para PMKID y WPS.
`sae_online.sh`	Ataque de diccionario *online* contra WPA3-SAE.
`guia_profesor.md`	Guía con la resolución y el fundamento técnico de cada escenario.
`MANUAL.md`	Este documento.

---

2. Requisitos

- **Máquina del profesor** (donde se genera el script): Python 3. Nada más.
- **VM de despliegue** (donde se ejecuta el laboratorio): Linux con `mac80211_hwsim` disponible (Kali es lo habitual). El propio script instala lo que falte (`hostapd`, `wpasupplicant`, `iw`, y según los escenarios `hcxtools`, `hostapd-wpe`, OpenCL para hashcat, etc.).
- **Diccionario** de claves accesible en la VM (por defecto `/usr/share/wordlists/rockyou.txt`; en Kali viene comprimido: `sudo gunzip`).
- Para los escenarios **PMKID** y **WPS**: hay que compilar una vez el `hostapd` parcheado (ver §7).

---

3. Flujo de trabajo en tres pasos

```
# 1. Editar la configuración del laboratorio
cp lab.conf.example mi_lab.conf
nano mi_lab.conf

# 2. Generar el script de despliegue (en la máquina del profesor)
./lab_generator.py mi_lab.conf -o despliegue.sh
```

```
# 3. Llevar a la VM y desplegar (como root)
sudo ./despliegue.sh
```

Al terminar, el script imprime un cuadro con el **SSID, canal y BSSID real** de cada escenario. Las interfaces de ataque del alumno son `wlan0`, `wlan1`, …

Para rehacer el laboratorio tras cambiar la configuración, vuelve a generar y a desplegar. Para apagarlo, usa tu `limpia.sh` (o reinicia la VM).

---

4. El fichero de configuración

Es un fichero de texto con dos tipos de líneas. Las líneas en blanco y lo que sigue a `#` se ignoran. Los campos se separan por **comas o espacios**, indistintamente.

4.1 Directivas globales (`clave = valor`)

Directiva	Valor por defecto	Significado
`dict`	`/usr/share/wordlists/rockyou.txt`	Diccionario del que salen las claves.
`student_ifaces`	`7`	Número de interfaces de ataque del alumno (`wlan0`…`wlanN-1`).

4.2 Líneas de red

```
ssid, tipo, clientes [, canal]
```

- **ssid**: nombre de la red (hasta 32 caracteres). Debe ser único.
- **tipo**: uno de los de la tabla de §5.
- **clientes**: número de estaciones que generan tráfico (ver límites en §5). En los tipos con clientes fijos (`pmkid_nc`, `wpa3_tm`, `wpa3_dg`) esta columna se ignora y se avisa.
- **canal** (opcional): número de canal. Si se omite, se usa el canal por defecto del tipo. La **banda** (2,4 o 5 GHz) se deduce del canal (ver §6).

4.3 Ejemplo

```
dict = /usr/share/wordlists/rockyou.txt
student_ifaces = 7

# ssid            tipo        clientes  canal
he_abierta        open        2
wpa2_casa         wpa2        2
pmkid_oficina     pmkid       1         11
corporativa       enterprise  1         3
```

```
router_wps      wps     1       6
wifi_moderna    wpa3    1       36
```

---

5. Tipos de escenario

tipo	Seguridad	Objetivo didáctico	Clientes (def / rango)	Canal def	hostapd
`open`	Abierta	Sniffing de tráfico HTTP en claro	1 / 0–4	1	sistema
`wpa2`	WPA2-PSK	Handshake 4-way + crackeo offline	2 / 0–4	6	sistema
`pmkid`	WPA2-PSK	PMKID **con** clientes + handshake	1 / 1–4	11	parcheado
`pmkid_nc`	WPA2-PSK	PMKID **sin** clientes (*clientless*)	0 (fijo)	4	parcheado
`enterprise`	WPA2-EAP	EAP-PEAP / MSCHAPv2 (Evil Twin)	1 / 0–4	3	sistema
`wep`	WEP-104	Recolección de IVs + reinyección ARP	2 / 0–4	9	sistema
`wps`	WPA2 + WPS	Fuerza bruta online del PIN	1 / 0–1	7	parcheado
`wpa3`	WPA3-SAE	Diccionario **online** contra SAE	1 / 0–4	5	sistema
`wpa3_tm`	WPA2/WPA3 mixto	Atacar la parte WPA2 (transición)	2 (fijo)	2	sistema
`wpa3_dg`	WPA3-SAE	Downgrade de cliente (Dragonblood)	1 (fijo)	10	sistema

- **"hostapd parcheado"** = necesita `hostapd-pmkid` (ver §7). El generador añade la comprobación automáticamente solo si usas alguno de esos tipos.
- La resolución y el fundamento técnico de cada escenario están en `guia_profesor.md`.

---

6. Canales y bandas (2,4 / 5 GHz)

La banda se deduce del número de canal; no hay que indicarla.

Banda	Canales válidos	Notas
2,4 GHz	`1`–`13`	`hw_mode=g`.

Banda	Canales válidos	Notas
5 GHz (no-DFS)	`36 40 44 48 149 153 157 161 165`	`hw_mode=a`, arranque instantáneo.
5 GHz (DFS)	`52 56 60 64 100 104 … 140 144`	El AP hace **CAC** (~1 min escuchando radar) antes de emitir.

Realismo para el alumno: al mezclar bandas, el alumno debe elegir la banda al atacar:

```
sudo airodump-ng --band a   wlan0mon   # solo 5 GHz
sudo airodump-ng --band abg wlan0mon   # 2,4 y 5 GHz
```

Sobre los canales DFS: se permiten, pero el AP tarda ~1 minuto en aparecer (hace el *Channel Availability Check*). El generador avisa al crear el script y el despliegue lo recuerda al arrancar ese AP; además espera más tiempo a que sus clientes conecten. Si quieres arranque inmediato, usa canales no-DFS.

Canales compartidos: puedes poner varias redes en el mismo canal (incluso la misma banda). El generador lo permite y lo avisa. Es útil para obligar al alumno a filtrar por **BSSID** en vez de por canal:

```
wpa2_casa     wpa2   1   6
invitados     open   1   6     # mismo canal -> hay que distinguir por BSSID
```

---

7. `hostapd` parcheado (PMKID y WPS)

Los tipos `pmkid`, `pmkid_nc` y `wps` necesitan un `hostapd` con dos parches:

1. Emitir el **PMKID** en el mensaje M1 también con WPA-PSK (el de serie no lo hace).
2. No exigir el **ACK** de la respuesta de autenticación/asociación, que en `mac80211_hwsim` no llega cuando el atacante inyecta desde modo monitor (afecta a `hcxdumptool`, `reaver` y `bully`).

Además, para WPS debe compilarse con `CONFIG_WPS=y`. Todo esto lo hace el script de compilación (una sola vez):

```
sudo ./compila_hostapd_pmkid.sh            # compila e instala
/usr/local/sbin/hostapd-pmkid
sudo ./compila_hostapd_pmkid.sh --force    # recompila aunque ya exista
```

El despliegue lo invoca solo si falta y hay escenarios que lo necesitan. El `hostapd` del sistema no se toca: el resto de escenarios lo siguen usando.

---

8. Las claves (anti-cheat)

- Las claves WPA2, WPA3, la contraseña del usuario Enterprise y la del POST de `open` se eligen **al azar del diccionario** en cada despliegue.
- La clave **WEP** son 13 caracteres aleatorios (el ataque WEP es estadístico, no de diccionario).
- El **PIN y la clave de WPS** son aleatorios propios del escenario.
- Las claves WPA2 se guardan como **PMK (hash)** en los `.conf`; las WPA3 van en claro porque SAE no admite PMK. Los `.conf` se **borran del disco** al terminar el despliegue.
- **Cambiar el repertorio de claves:** pasa otro diccionario al desplegar (tiene prioridad sobre la directiva `dict`):

```
sudo ./despliegue.sh /ruta/otro_diccionario.txt
```

Como las claves cambian en cada despliegue, **no hay tabla fija de soluciones**. El profesor las consulta con `hostapd_cli`/`wpa_cli` (ver `guia_profesor.md`, sección "Consultar las claves del despliegue actual") o, simplemente, crackeando con el mismo diccionario, que es el propio ejercicio. Para que el crackeo sea reproducible en clase, el alumno debe usar **ese mismo diccionario**.

---

9. Límites y dimensionado

- **Tope del módulo:** 100 radios (`mac80211_hwsim`). El generador lo comprueba y aborta con un mensaje claro si la configuración lo supera.
- **Radios por escenario:** 1 (AP) + nº de clientes (+1 el cliente efímero de `pmkid_nc`). Total = `student_ifaces` + suma por escenario.
- **Aviso de recursos:** por encima de 50 radios, el generador avisa de que la VM necesitará RAM/CPU suficientes. Como referencia, 4 GB van cómodos para ~30 radios; sube a 6–8 GB para el doble. Los bucles de tráfico (`ping`, `curl`) consumen CPU: bájalos si hace falta.

---

10. Referencia de comandos del generador

```
./lab_generator.py CONFIG [-o SALIDA] [--dict RUTA]
```

Argumento	Descripción
`CONFIG`	Fichero de configuración del laboratorio (obligatorio).
`-o`, `--output`	Script de salida (por defecto `despliegue.sh`).
`--dict RUTA`	Sobrescribe la directiva `dict` del fichero.

Ejemplos:

```
./lab_generator.py mi_lab.conf
./lab_generator.py mi_lab.conf -o lab_examen.sh
./lab_generator.py mi_lab.conf --dict /usr/share/wordlists/fasttrack.txt
```

El generador valida la configuración antes de escribir nada y, si algo no cuadra, sale con un error explicativo sin generar el script (ver §12).

---

11. Despliegue en la VM

```
sudo ./despliegue.sh                       # claves de rockyou.txt
sudo ./despliegue.sh /ruta/diccionario.txt # claves de otro diccionario
```

El script, por orden: instala paquetes que falten, carga `mac80211_hwsim` con el número exacto de radios, renombra las interfaces, crea los namespaces, genera las configuraciones (con claves del diccionario), arranca los APs y los clientes con su tráfico, imprime el resumen y borra los `.conf` del disco.

Poner una interfaz del alumno en modo monitor:

```
sudo ip link set wlan0 down
sudo iw dev wlan0 set type monitor
sudo ip link set wlan0 up
```

> No uses `airmon-ng check kill`: mataría los clientes del laboratorio. El script ya excluye las radios virtuales de NetworkManager.

---

12. Mensajes de error del generador

Todos empiezan por `[-] Error de configuración:` e indican la línea:

Mensaje	Causa y solución
`tipo desconocido '...'`	Tipo mal escrito. Usa uno de §5.
`SSID duplicado '...'`	Dos redes con el mismo nombre. Renómbralas.
`'tipo' admite entre X e Y clientes`	Número de clientes fuera de rango.
`canal N no válido`	Canal inexistente. Usa uno de §6.
`el laboratorio necesita N radios …`	Supera el tope de 100. Reduce redes/clientes.
`'wlan...' supera 15 caracteres`	Demasiados escenarios/clientes (límite de nombres de Linux).

Avisos (no detienen la generación), por `stderr`:

- `[aviso] … usa un número de clientes fijo` — tipo con clientes fijos.

- `[aviso] canal N es DFS …` — el AP tardará ~1 min (CAC).
- `[aviso] el laboratorio usa N radios (> 50) …` — vigila la RAM/CPU.
- `[info] canal N compartido por …` — varias redes en el mismo canal.

---

13. Problemas frecuentes en el despliegue

Síntoma	Causa probable	Solución
`Falta hostapd-pmkid …`	No compilado	`sudo ./compila_hostapd_pmkid.sh`
`hostapd-pmkid sin CONFIG_WPS=y`	Compilado sin WPS	Recompila con `--force` (el script ya añade `CONFIG_WPS=y`).
`El hostapd del sistema no tiene soporte SAE`	hostapd antiguo	Actualiza `hostapd` (en Kali reciente viene con SAE).
`No se puede leer el diccionario`	Ruta mala o comprimido	`sudo gunzip rockyou.txt.gz` o pasa otra ruta.
Un AP de 5 GHz no aparece	Regulatorio / DFS	Comprueba `iw reg get`; en DFS espera ~1 min; valida con `wireless-regdb`.
`reaver` se queda en `Receive timeout`	AP sin el parche del ACK	Asegúrate de que `wps` usa `hostapd-pmkid` (lo hace el generado).

---

14. Un ejemplo completo de principio a fin

`examen.conf`:

```
dict = /usr/share/wordlists/rockyou.txt
student_ifaces = 7

he_wpa2_facil     wpa2        1
he_pmkid_nc       pmkid_nc    0
he_wps_lab        wps         1
he_wpa3_5g        wpa3        1   36
he_mixto_a        wpa2        1   6
he_mixto_b        open        1   6
```

```
# Profesor
./lab_generator.py examen.conf -o examen.sh
#   [info] canal 6 compartido por: he_mixto_a, he_mixto_b (filtrar por BSSID)
#   [OK] Generado examen.sh (6 escenarios, 18 radios)

# VM (una vez, si hay pmkid/wps)
sudo ./compila_hostapd_pmkid.sh
```

```
# VM (cada despliegue)
sudo ./examen.sh
```

El alumno recibe las interfaces `wlan0`…`wlan6`, localiza objetivos en ambas bandas con `airodump-ng --band abg`, y resuelve cada escenario según `guia_profesor.md`.